Software Engineering

Dr. Fatma ElSayed

Dr. Ahmed Yousry

Computer Science Department

Course Information

Contents

- Introduction to Software Engineering
- Software Processes
- Requirements Engineering
- System Modelling Part 1
- System Modelling Part 2
- System Modelling Part 3
- System Architecture
- Software Testing Strategies
- Software Testing Techniques
- Technical Metrics for Software

Course Information

Contents

- Introduction to Software Engineering
- Software Processes
- Requirements Engineering
- System Modelling Part 1
- System Modelling Part 2
- System Modelling Part 3
- System Architecture
- Software Testing Strategies
- Software Testing Techniques
- Technical Metrics for Software

Chapter 5: System Modelling

Sequence Diagram

Interaction Models

- All systems involve interaction of some kind. This can be:
  - User interaction, which involves user inputs and outputs *(helps to identify user requirements).*
  - Interaction between the system being developed and other systems (highlights the communication problems that may arise).
  - Interaction between the components of the system *(helps us understand if a proposed system structure is likely to deliver the required system performance and dependability).*

Two related approaches to interaction modelling:

1. **Use case modelling,** which is mostly used to model interactions between a system and external actors.
2. **Sequence diagrams,** which are used to model interactions between system components, although external agents may also be included.

Sequence Diagram

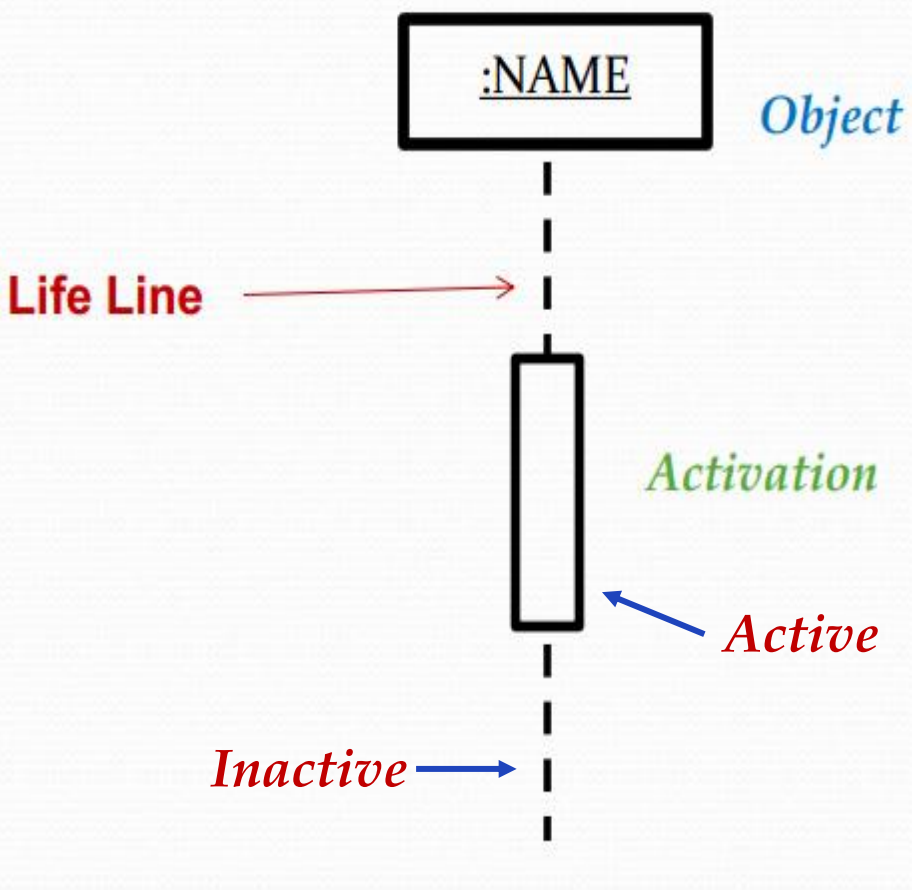
- A **sequence diagram** shows the sequence of interactions that take place during a particular use case to fulfill a specific task.
- Each diagram represent one of the flows through a **use case.**
- Each Use Case requires one or more sequence diagrams to describe its behavior.
- Sequence diagrams are interaction diagrams **ordered by time.**

Shows, step-by-step, flows through a use case:

- What **objects** are needed for the flow.
- What **messages** the objects send to each other.
- What **actor** initiates the flow.
- What **order** the messages are sent.

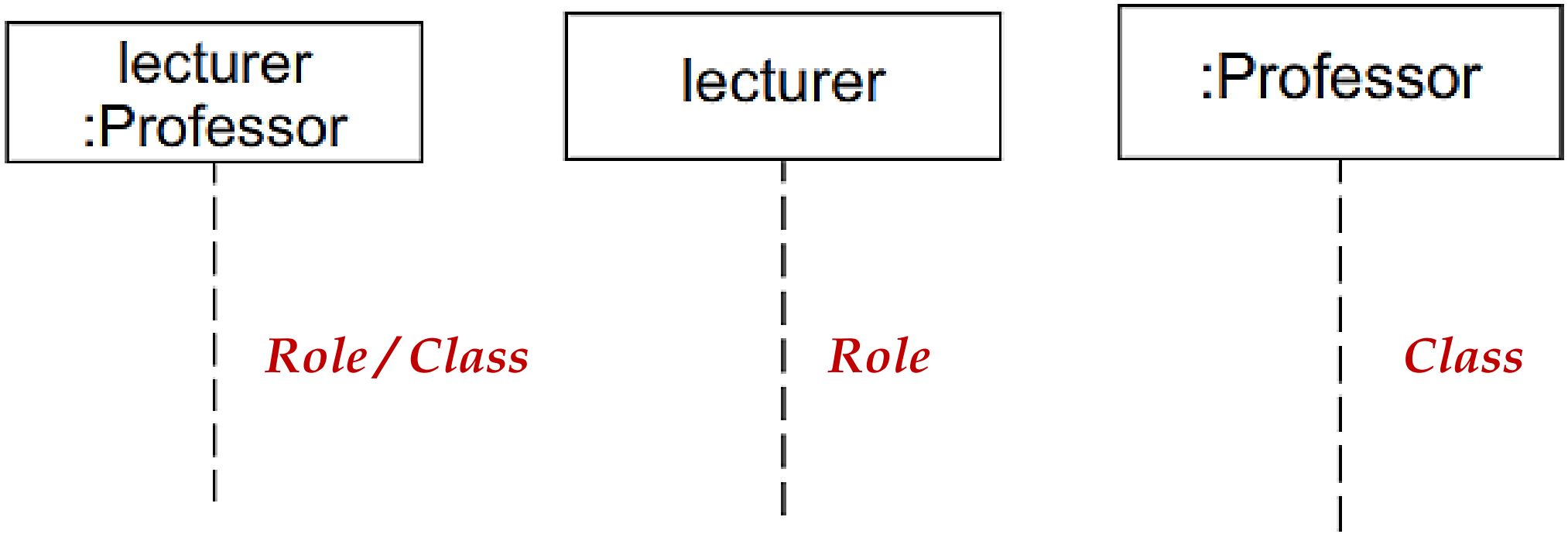
Object Lifeline

- A **lifeline** is shown as a vertical, usually dashed line that represents the lifetime of the object associated with it.
- At the top end of the line is the **head** of the lifeline, a rectangle which contains an expression in the form ***roleName:Class.***
- This expression indicates the **role** name and the **class** of the object associated with the lifeline



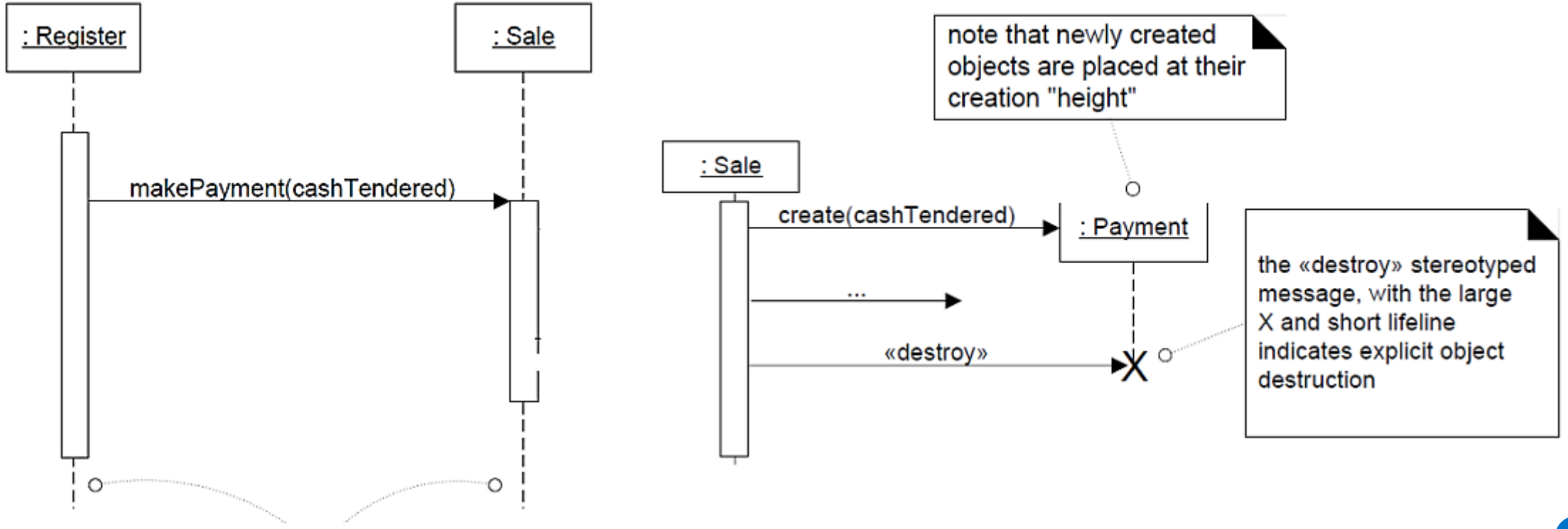
Types of Object Lifeline

- If the class is omitted, the colon can also be omitted; if only the class is specified, it must be preceded by a colon.



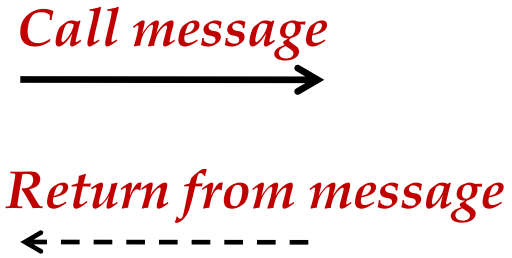
Object Lifeline

- The sequence diagram is a two-dimensional diagram.
  - The top-to-bottom dimension shows the passage of time. ***[vertical]***
  - The left-to-right dimension is the layout of the objects. ***[horizontal]***



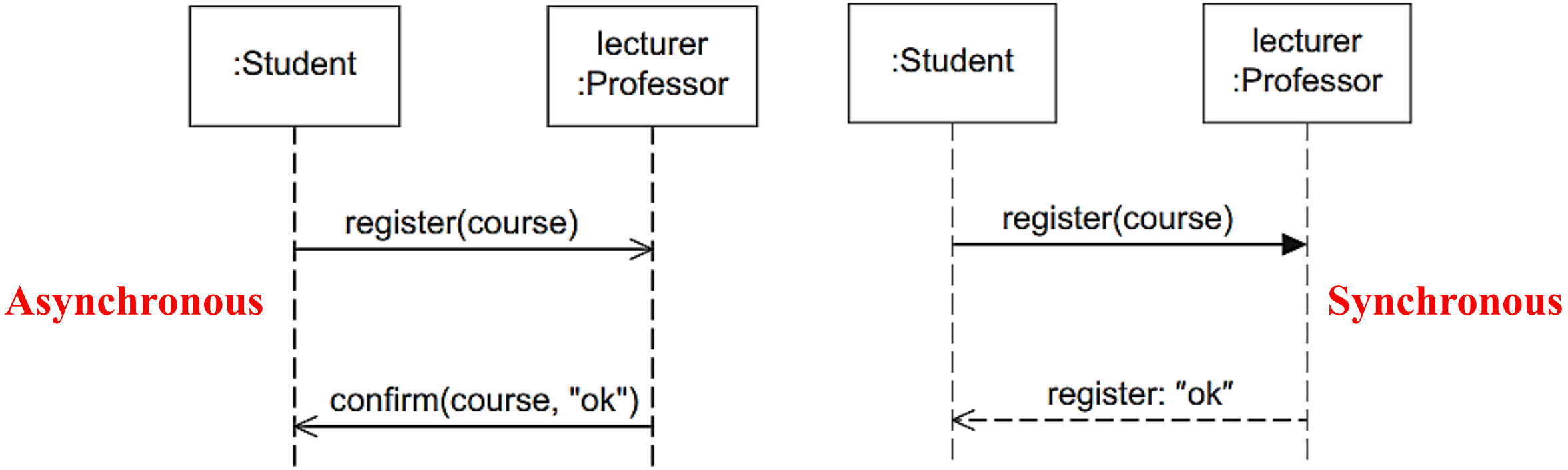
Exchanging Messages

- A message is shown as an arrow from the **sender** to the **receiver**.
- The type of the arrow expresses the type of communication involved, and there are two types of messages:
  1. A ***synchronous*** message is represented by an arrow with a continuous line and a **filled triangular arrowhead**.
     - The sender **waits** until it has received a response message before continuing.
     - The response message is represented by a dashed line with an open arrowhead.



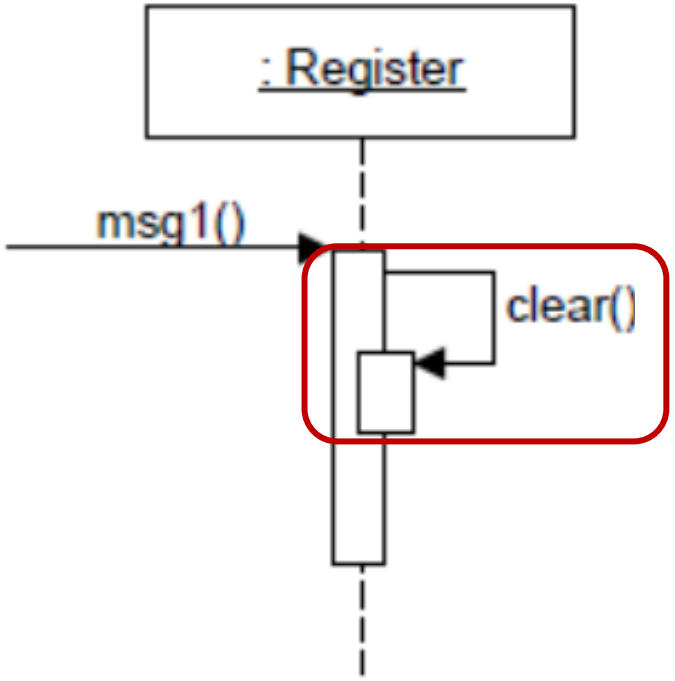
Exchanging Messages

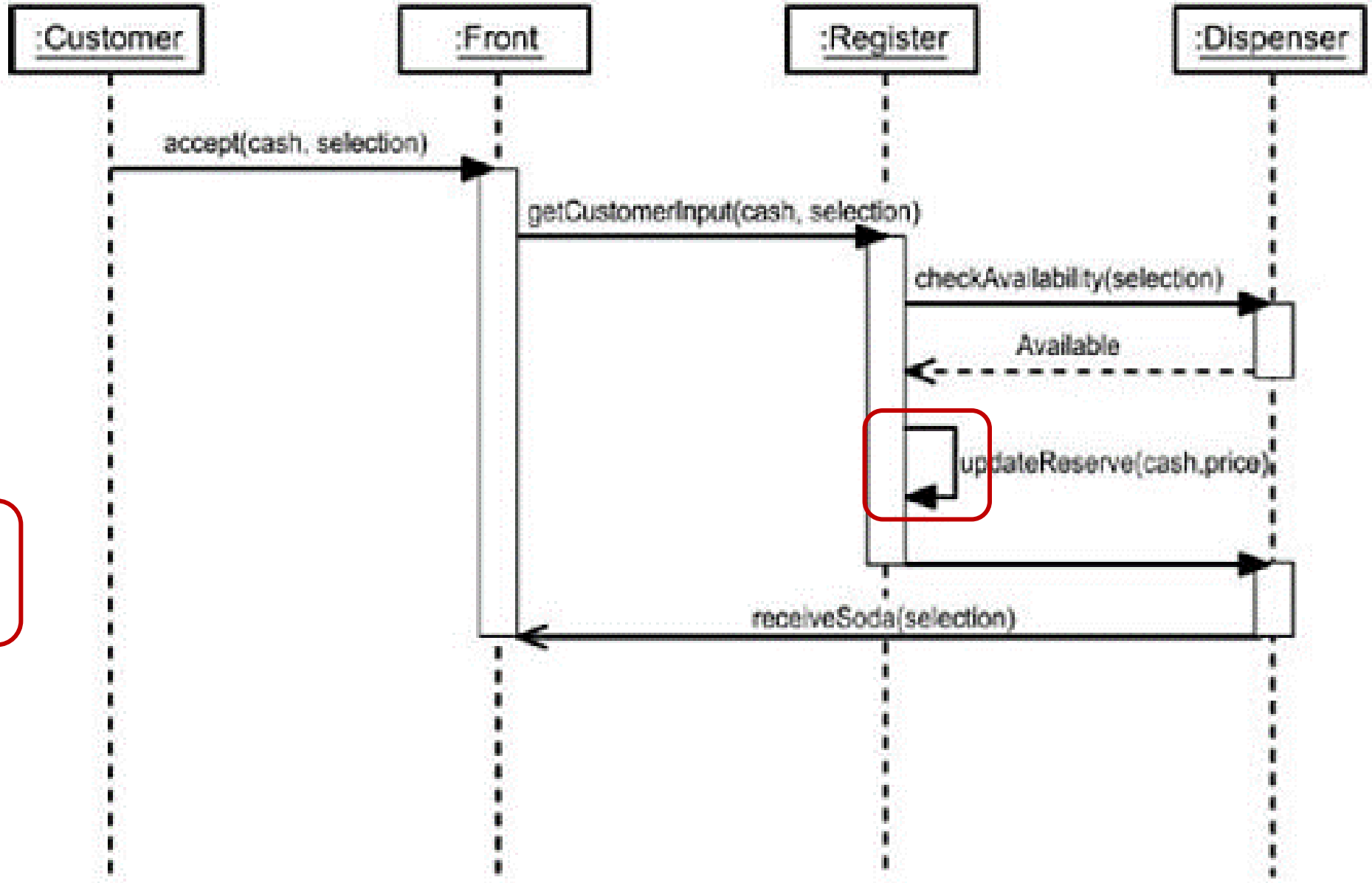
2. An **asynchronous** message is represented by an arrow with a continuous line and an **open arrowhead**.
   - The sender continues after having sent the message.



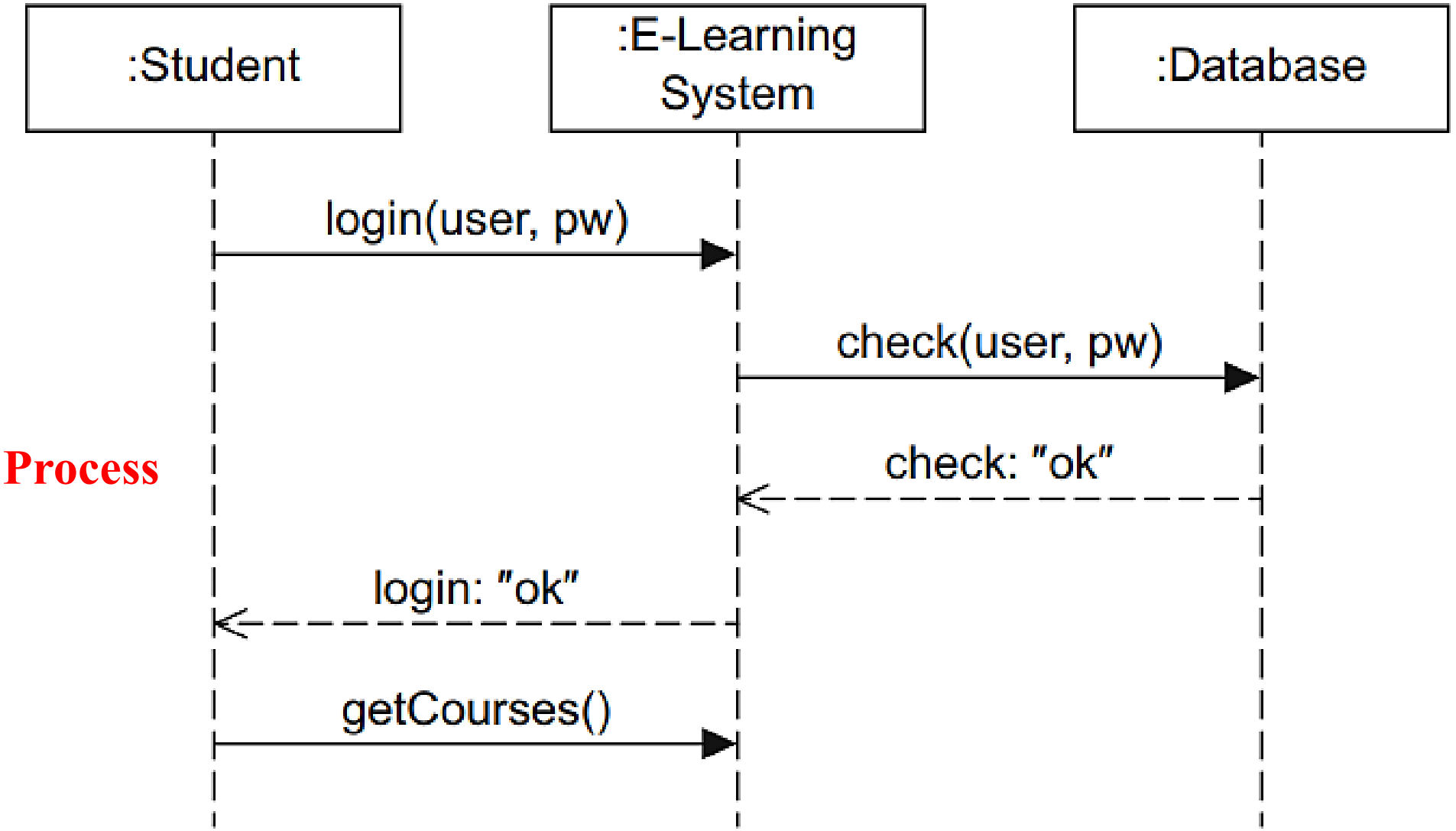
Register for a course

Message to Self





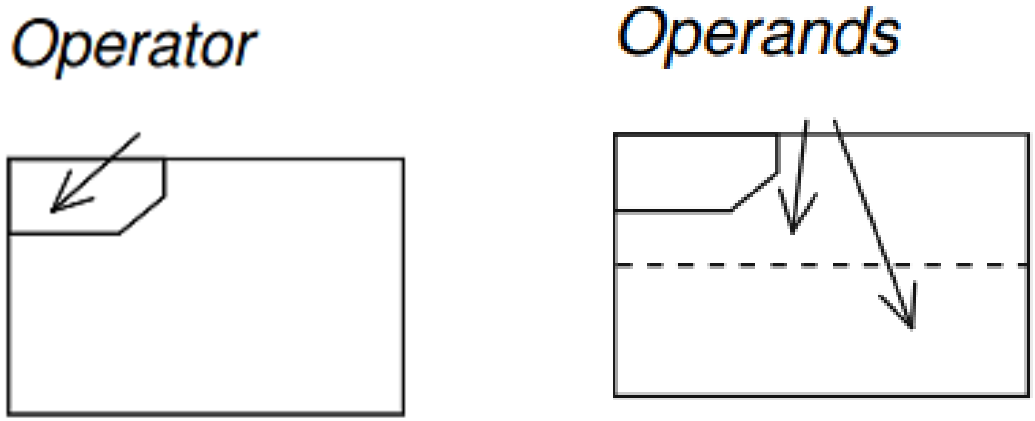
Sequence Diagram: Example



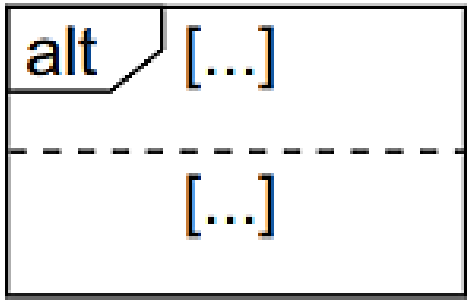
Login Process

Alternatives and Iterations

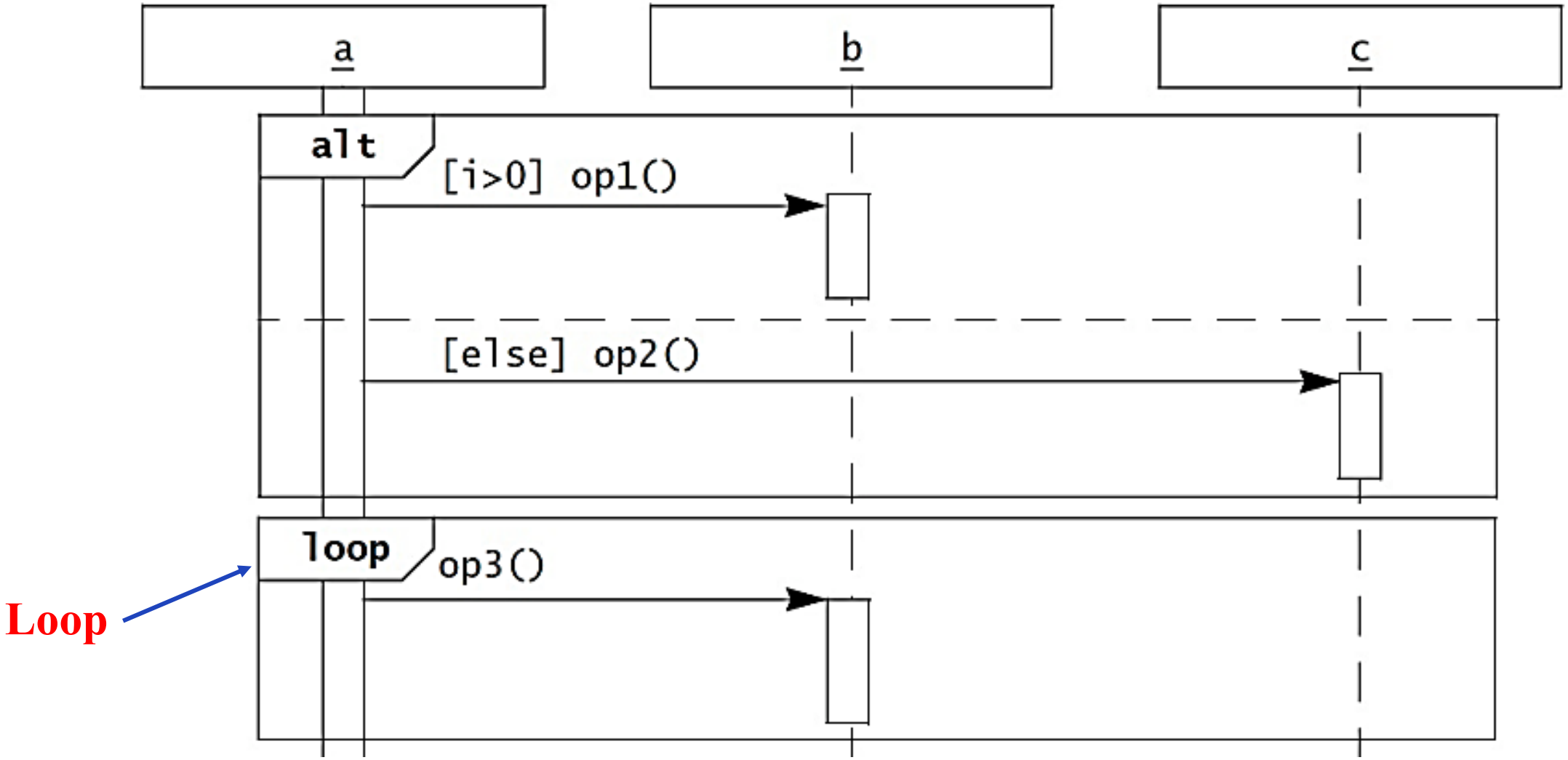
- In a sequence diagram, you can use ***combined fragments (operators)*** to describe a number of possible execution paths.



- You can use an **alt** fragment to represent alternative sequences, each operand represents an alternative path in the execution.
- Each operand has a ***guard***.



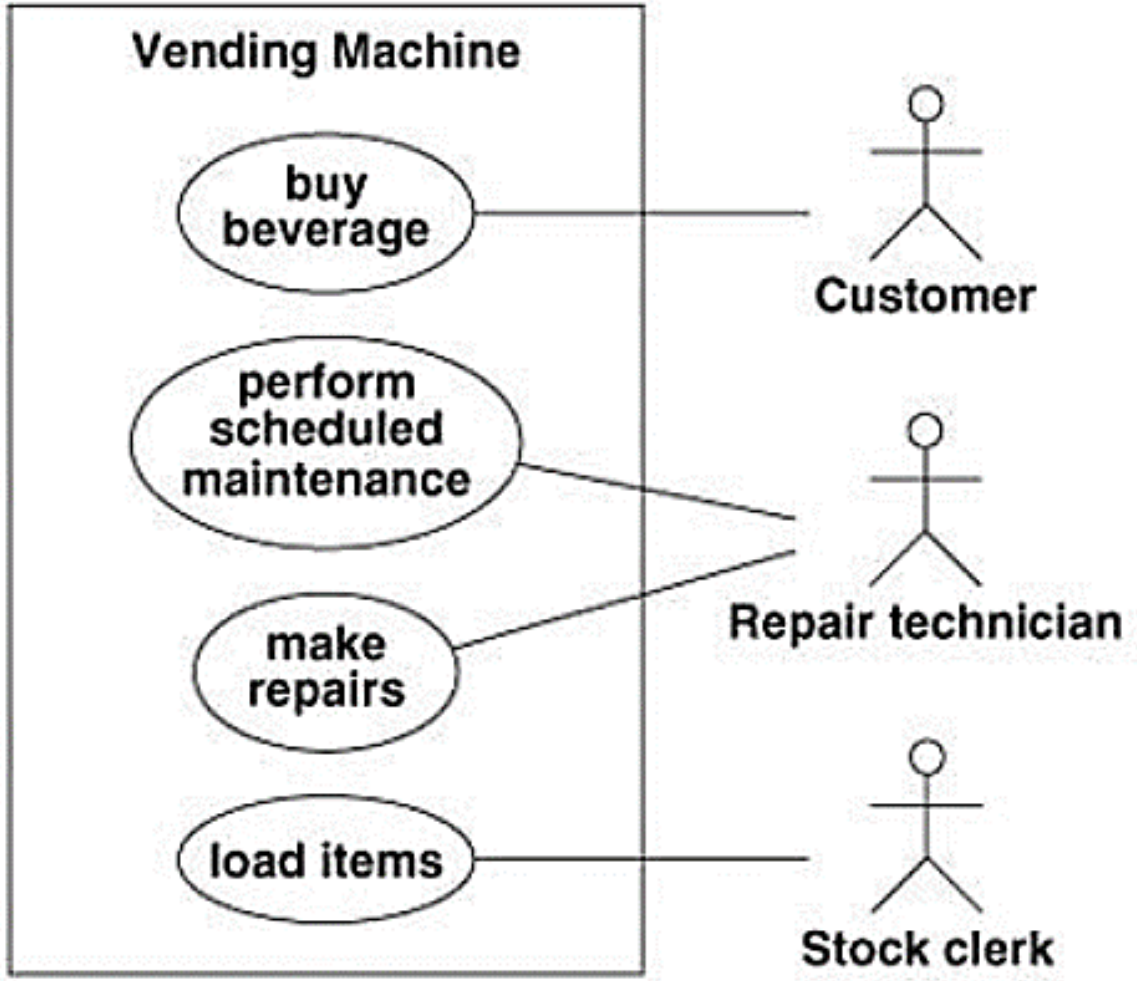
Alternatives and Iterations: Example



System Sequence Diagram (SSD)

For a use case scenario, an SSD shows:

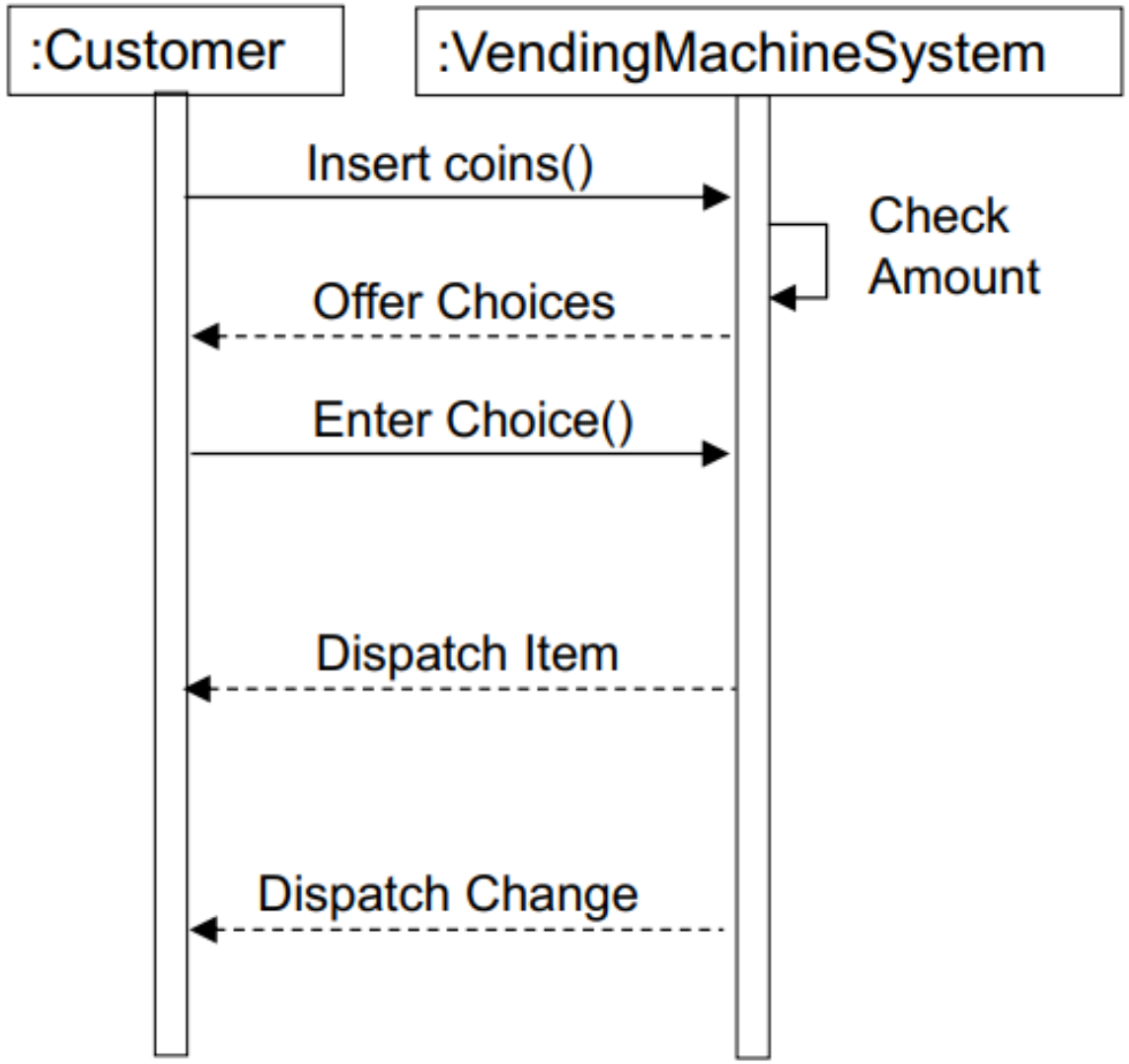
- The System (as a black box).
- The external actors that interact with System.
- The System events that the actors generate.



System Sequence Diagram (SSD)

The System events are:

- Insert coins()
- Enter choices()



Class Diagram

Structural Models: Class Diagram

- Class diagrams are used when developing an **object-oriented** system model to show the **classes** in a system and the **associations** between these classes.
- A class model captures the **static structure** of the system by characterizing:
  - The **classes and objects** in the system,
  - The **attributes** and **operations** for each class of objects, and
  - The **relationships** among the objects.
- Class models are the **most widely used UML diagram**.
- In OO systems we build the system around objects, not functionality.

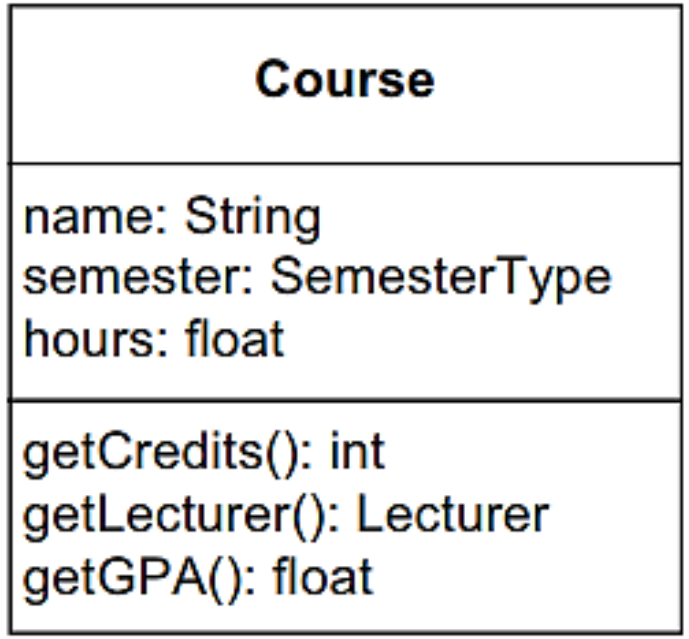
Objects

- **Objects** represent something in the **real world**, such as a patient, a course, a doctor, etc.
- Objects often appear as **proper nouns** in the problem description or discussion with the customer.
- The choice of objects depends on the analyst's judgment and the problem at hand. There can be more than one correct representation.

- A class is the description of a **set of objects** *(having similar attributes, operations, relationships, and behavior)*. while **Objects** are instances of classes.

Objects

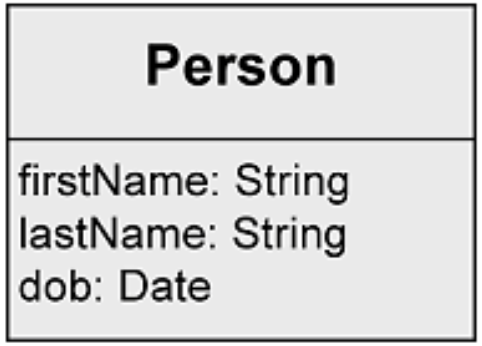
Class name



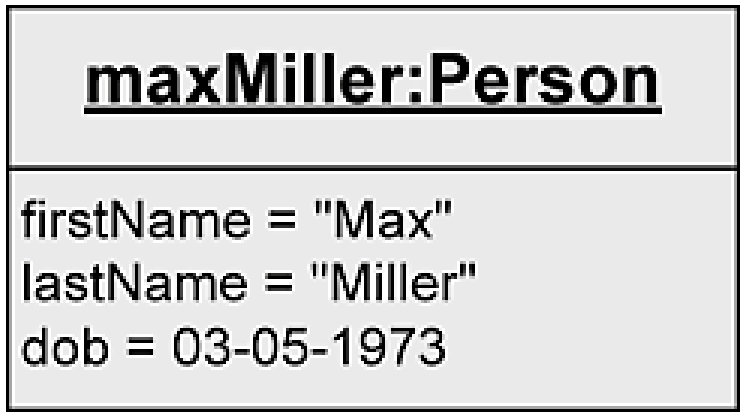
Attributes

Operations

Class



An object of that class



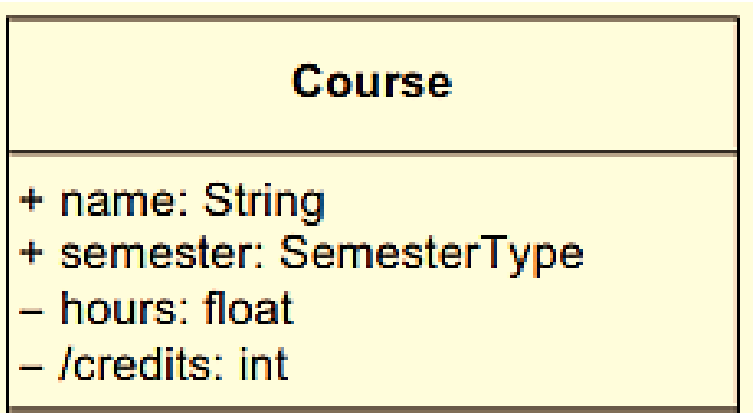
Class Attributes

- An attribute allows you to **store** information that is known for all instances.
- Attribute declaration may include visibility, type and initial value

Who is permitted to access the attribute

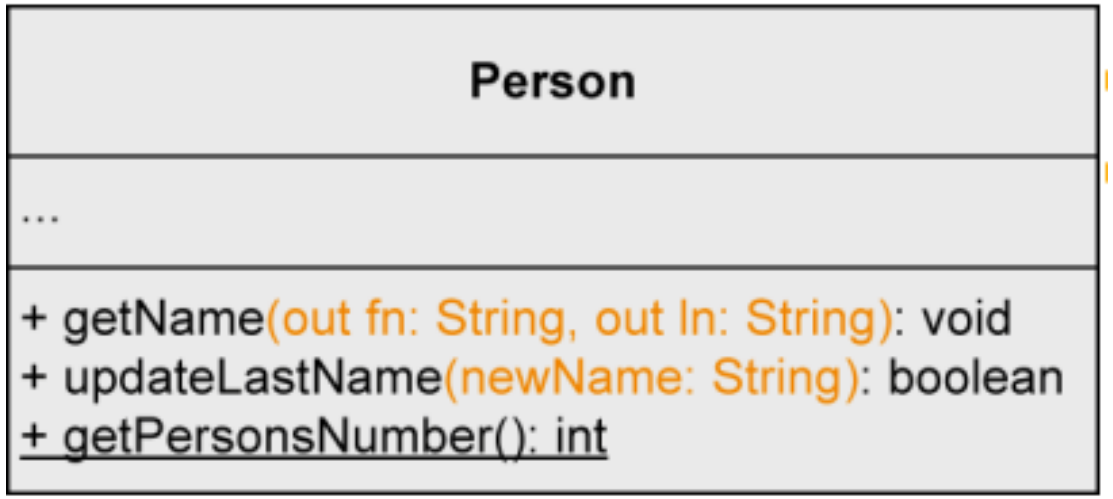
- \+ ... **public**: everybody
- \- ... **private**: only the object itself
- # ... **protected**: class itself and subclasses

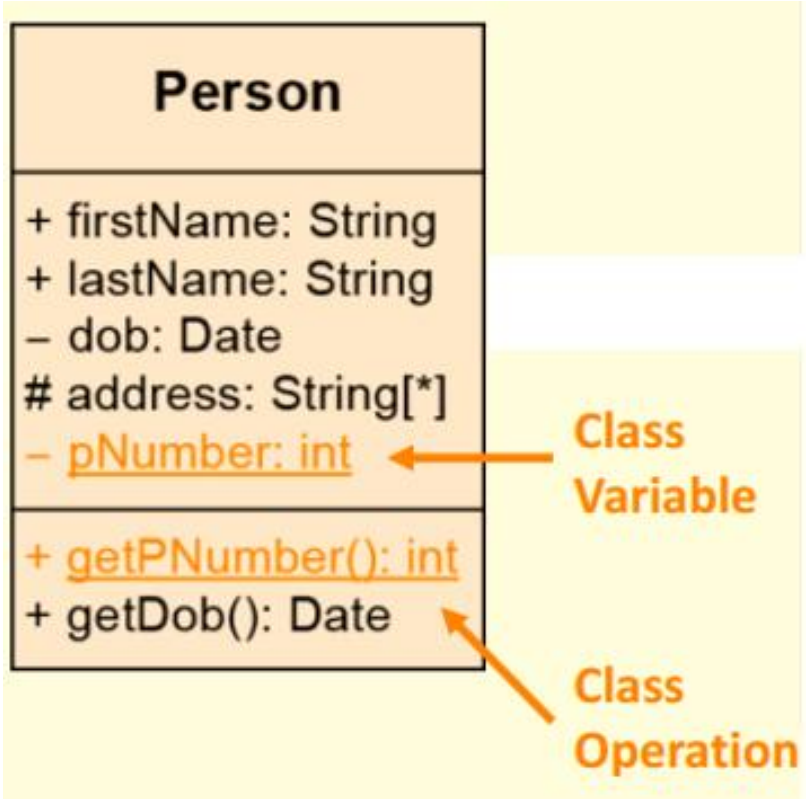
visibility



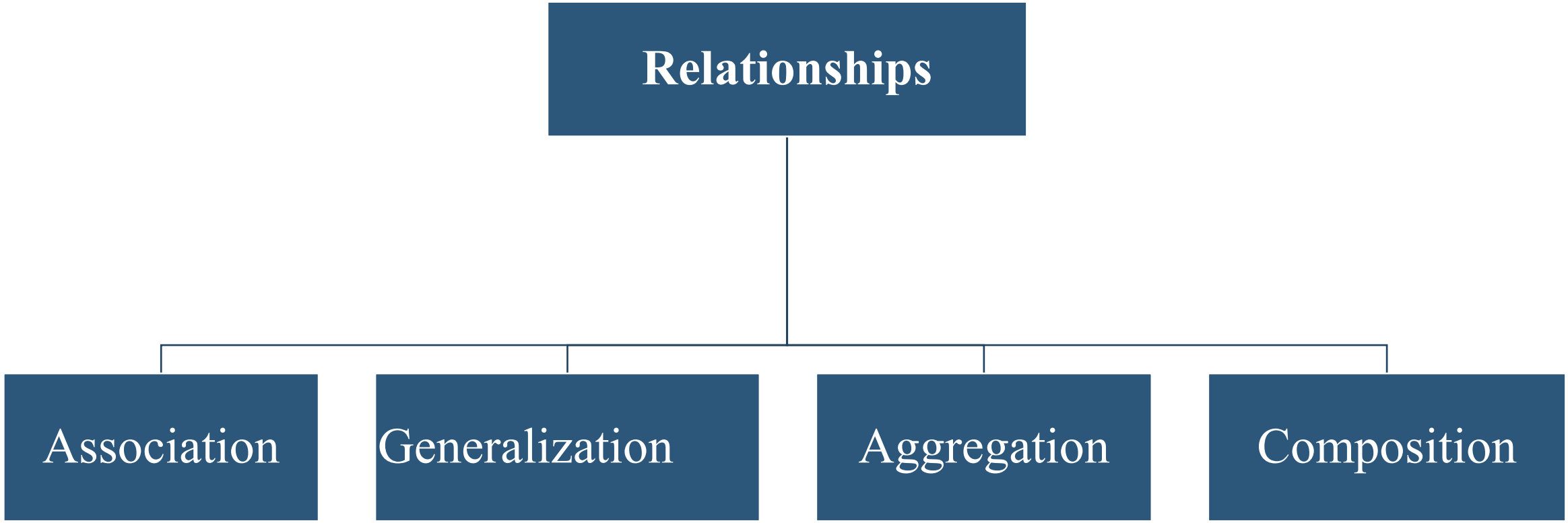
Class Attributes & Operations

- Operations enable objects to **communicate** with one another and to act and react.
- An operation may include visibility, parameters, and return type:





Relationships

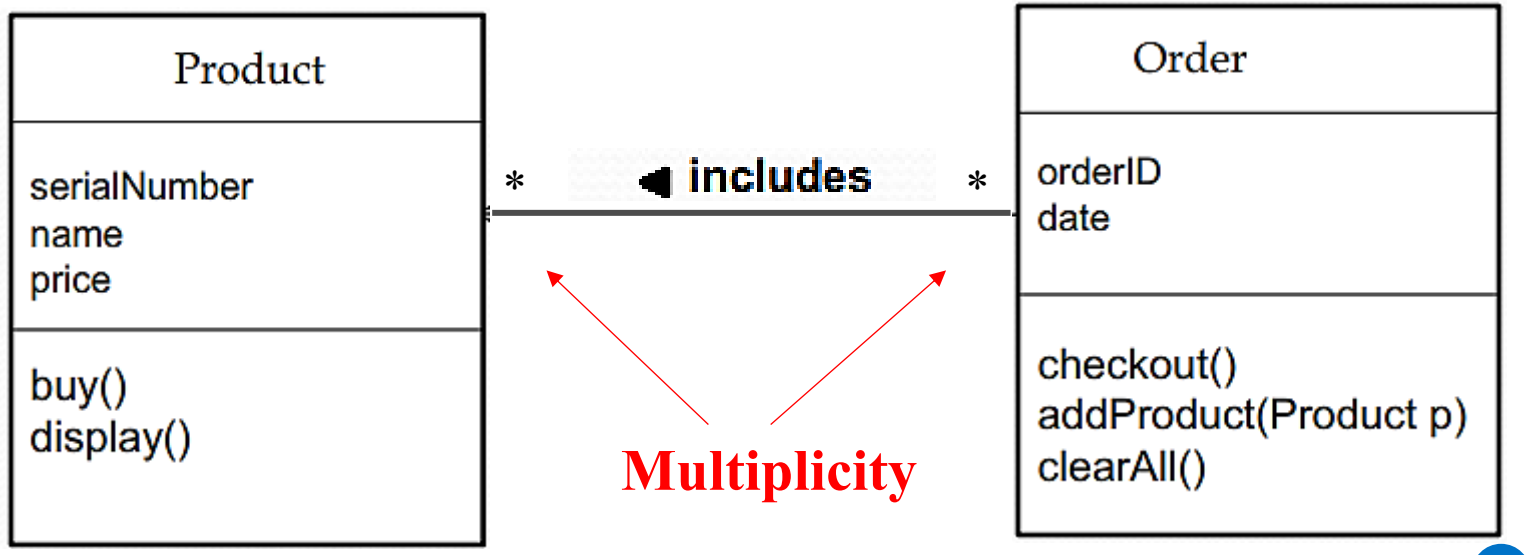


Associations

- An association is a **relationship** between classes and represented by a **line** with a name.

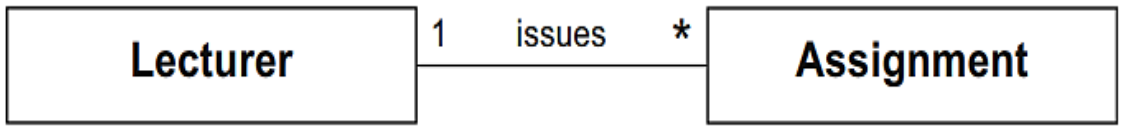


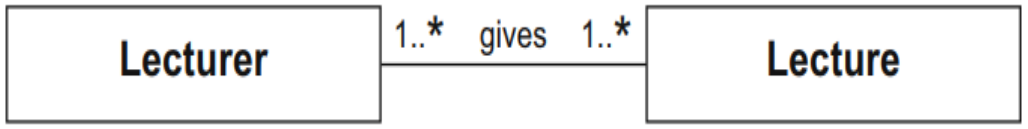
- An **optional** "reading direction arrow" indicates the **direction** to read the association name.



Multiplicity

- **Multiplicity:** The Number of objects that may be associated with exactly one object of the opposite side.







Zero or more



One or more



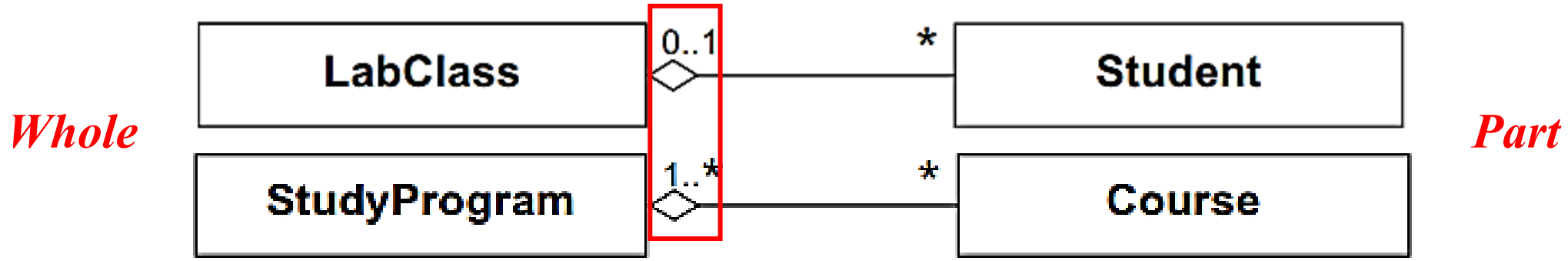
Exactly 5



One to 40

Aggregation

- An aggregation is a special ***form of association*** that is used to express that instances of one class **are parts of** an instance of another class.
- Represented by a **diamond** at the association end of the class that stands for the ***"whole"***.
- Expresses a **weak belonging** of the parts to a whole, meaning that parts also exist **independently** of the whole.



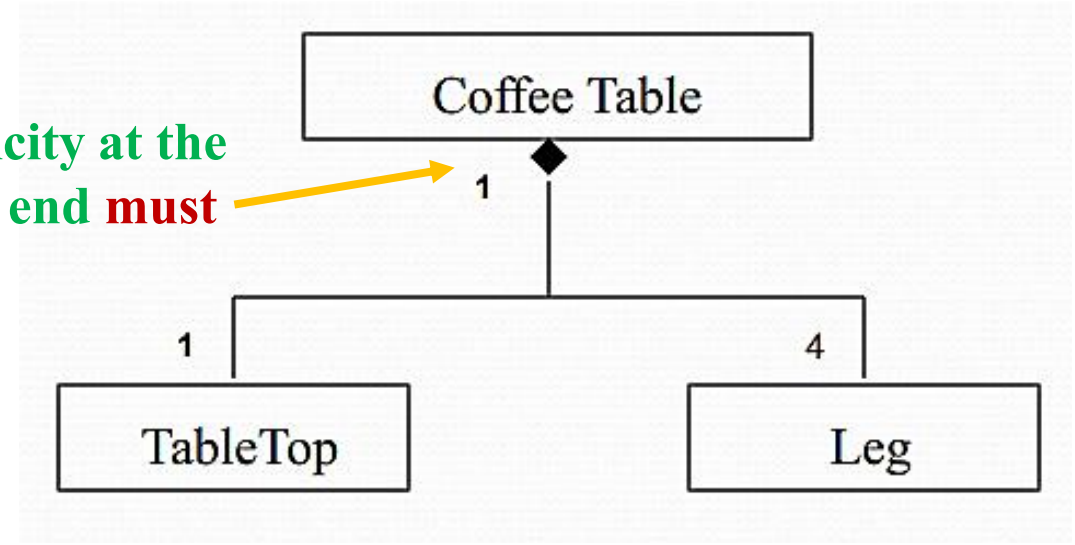
The multiplicity at the aggregating end may be greater than 1

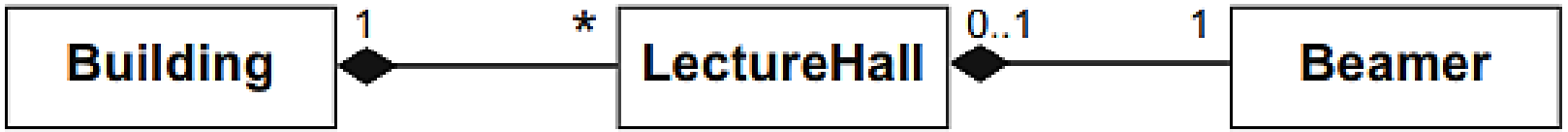
Composition

A <u>strong</u> form of aggregation

- The part object may belong to ***only one*** whole
- The life time of the part is ***dependent*** upon the whole.
- The composite must manage the ***creation*** and ***destruction*** of its parts.

The multiplicity at the composition end must be zero or 1

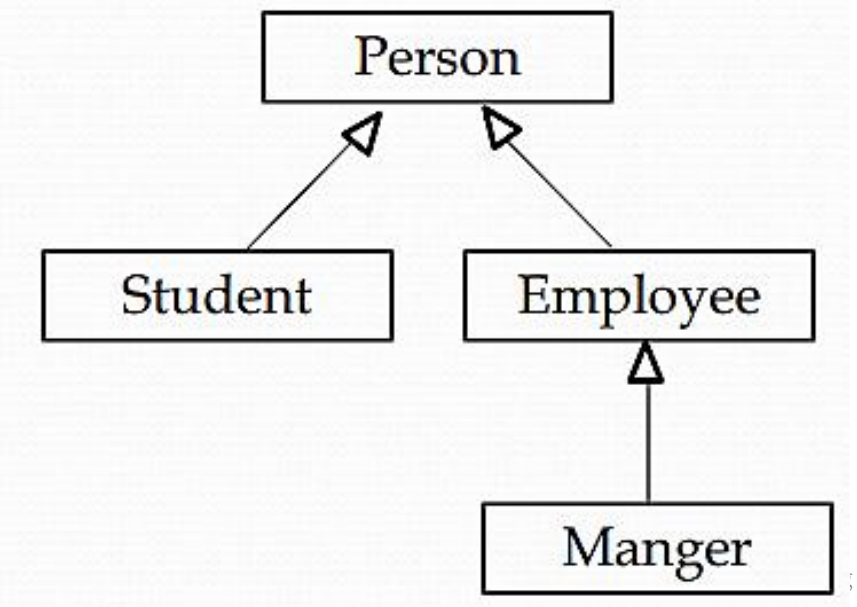




Generalization

- The generalization relationship is also referred to as **inheritance**.
  1. A class might have no parents, which is a **base class or root class**.
  2. A class might have no children, in which case it's a **leaf class**.
  3. If a class has exactly one parent, it has a **single inheritance**.
  4. If a class has more than one parent, it has **multiple inheritance**.

*Inheritance is called **"is-a"** relationship.*



Thank You